Introduction to Computer Science and Programming

Labo 5 – Exercises about Von Neumann and Assembly

Laboratory 5, November 5, 2024

Filippo Ricca

DIBRIS, University of Genova

Today…

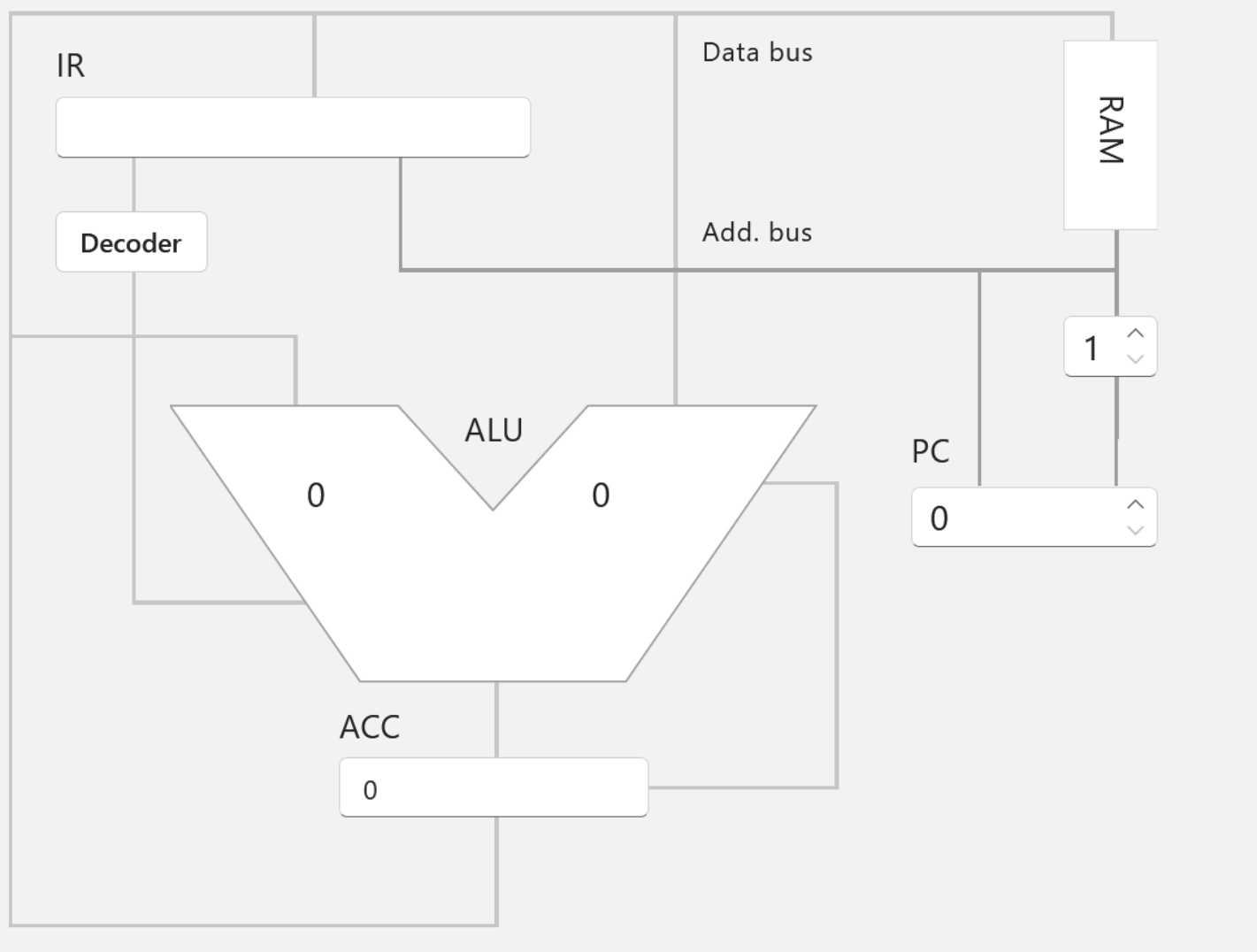
Today's lab is dedicated to the **Von Neumann Simulator**

1. Today we'll review (quickly) the **main components of how a Von Neumann machine works**, such as the IP, PC, ALU, and the fetch-decode-execute cycle
2. Then, we'll look at the various **commands that can be used**
3. Then, we'll do some **simple examples in Assembly** using the simulator
   - https://vnmsim.c2r0b.ovh/en-us
4. Finally, you will complete some exercises that you will need to submit

The CPU uses the **address bus** to specify the location in memory (or in an I/O device) it wants to access

The **data bus** transports data between the CPU, memory, and peripheral devices. For example, if the CPU reads a value from memory, that value will travel over the data bus

IR stores the instruction that is currently being executed



RAM contains program and the data

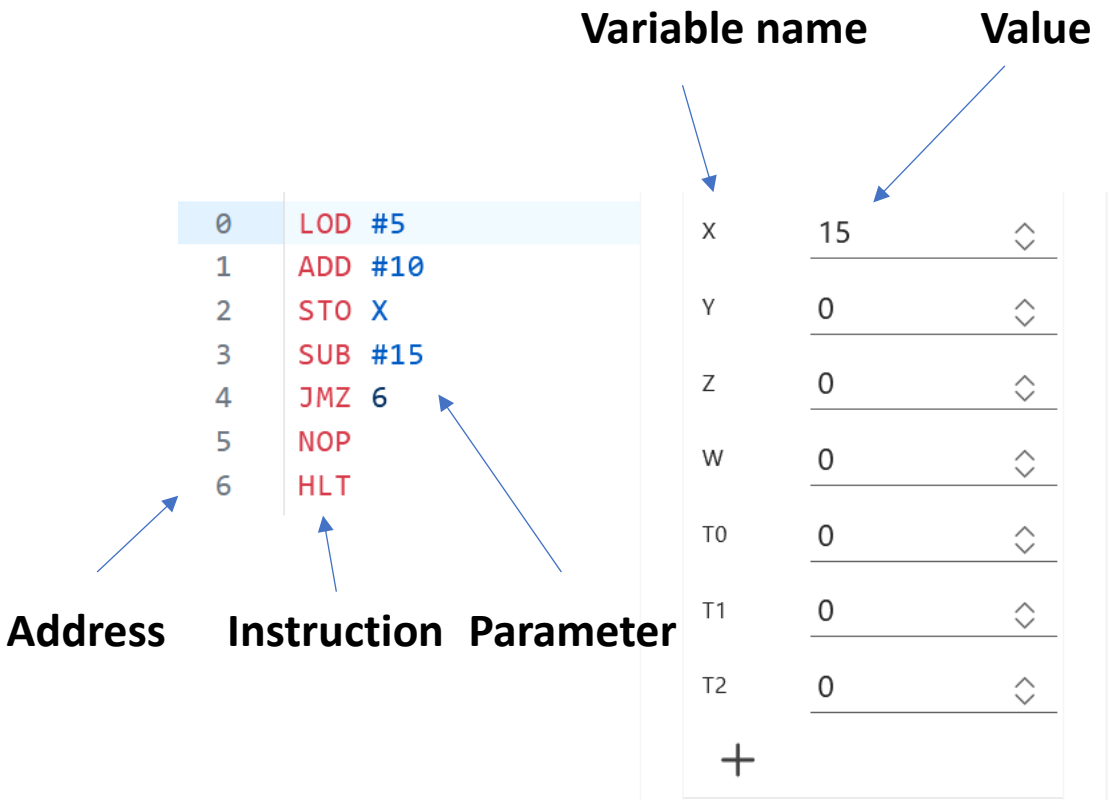
ALU Implements Arithmetic and Logical Operations

PC keeps the memory address of the next instruction to execute

ACC register stores intermediate arithmetic logic unit results

Von Neumann Machine Simulator

Program and data (RAM) in the Von Neumann Simulator



Variables

It is possible to change the value

X	2
Y	2
Z	0
W	0
T0	0
T1	0
T2	0

+

A variable can only contain positive or negative integer values

It is possible to add a novel variable (e.g., T3)

Menu and commands

Create a blank new project

Assembly examples

Commands Syntax and Semantics

Execute and see step by step

One step execution (and then stop)

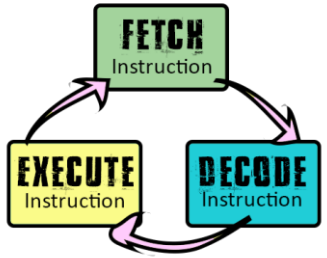
Execute and show final result

Rotate during execution

Stop execution (e.g., infinite loop)

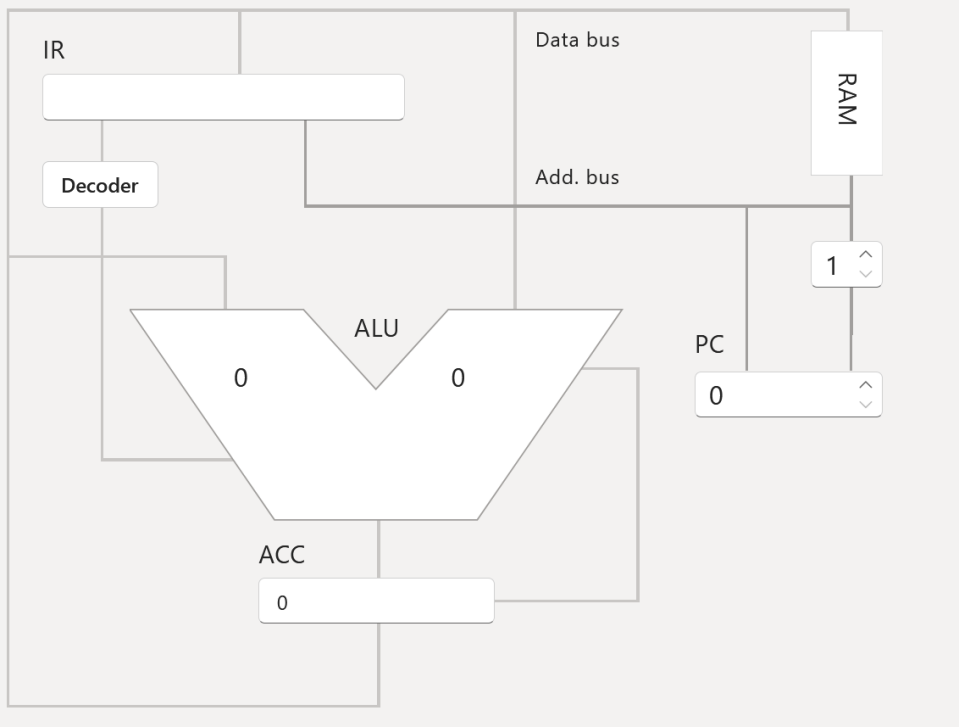
Execution speed

Fetch-Decode-Execute cycle

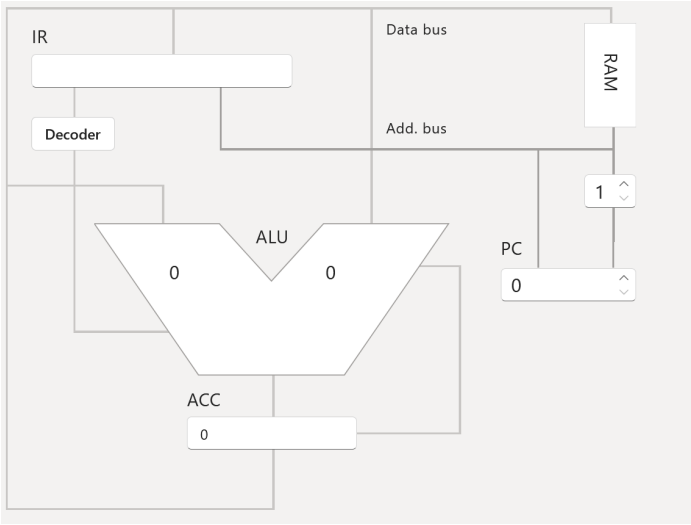


Algorithm:

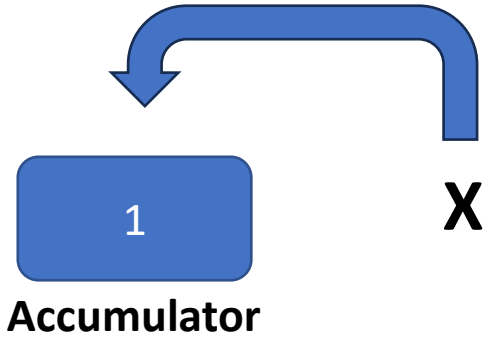
1. **Reset**: Insert the value 0 into the PC register
2. **Fetch**: Retrieve the value from memory RAM[PC] and store it in the IR register.
   - Then, increment the value stored in the PC register by 1
3. **Decode**: Observe the value in the IR register and interpret it as the representation of an instruction to execute
4. **Execute**: Execute the instruction decoded in the previous step.
   - Once the required operations are completed, return to the 'fetch' phase, unless the instruction to be executed was the program termination instruction



Assembly commands



LOD (Load) and STO (Store)

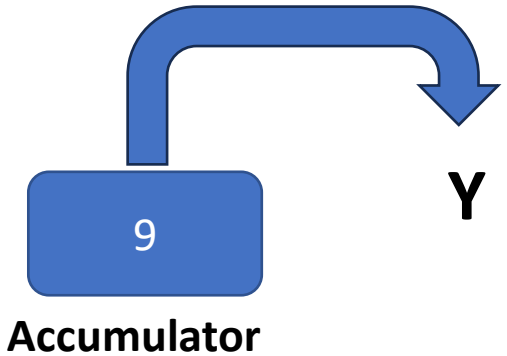


- **LOD: Loads a value into the accumulator**
- **Syntax:**
  - LOD var – loads the value stored at the memory location var into the accumulator
  - LOD #num – loads the literal value num directly into the accumulator

```
LOD #5      // Loads the literal value 5 into the accumulator
LOD X       // Loads the value stored at memory location X into the accumulator
```

- **STO: Stores the value in the accumulator into a specified memory location**
- **Syntax:**
  - STO var – stores the accumulator's current value into memory location var

```
STO Y       // Stores the accumulator's value in memory location Y
```



Arithmetic Operations

- **ADD, SUB, MUL, DIV: are used to do an operation between a variable and the accumulator**

```
ADD Z       // Adds the value at memory location Z to the accumulator
ADD #10     // Adds 10 to the accumulator
```

```
SUB W       // Subtracts the value at memory location W from the accumulator
SUB #3      // Subtracts 3 from the accumulator
```

```
MUL Q       // Multiplies the accumulator by the value at memory location Q
MUL #4      // Multiplies the accumulator by 4
```

```
DIV R       // Divides the accumulator by the value at memory location R
DIV #2      // Divides the accumulator by 2
```

JMZ (Jump if Zero) and JMP (Unconditional Jump)

- **JMZ: Jumps to a specified memory location if the accumulator's value is zero**
- Syntax: JMZ cell – if the accumulator is zero, the program counter moves to the memory location cell

`JMZ 10` // Jumps to instruction at cell 10 if the accumulator is zero.

- **JMP: Unconditionally jumps to a specified memory location**
- Syntax: JMP cell – moves the program counter to the specified memory location cell, regardless of the accumulator's value.

`JMP 20` // Jumps to instruction at cell 20.

NOP (No Operation) and HLT (Halt)

- **NOP: Does nothing for one cycle; useful for alignment or to introduce delay**
- Syntax: NOP

```
NOP        // Executes no operation and moves to the next instruction
```

- **HLT: Stops the program execution**
- Syntax: HLT

```
HLT        // Stops program execution
```

Quick demo

Example Program: Simple Addition and Conditional Jump

```
0: LOD #5          // Load 5 into the accumulator
1: ADD #10         // Add 10 to the accumulator, resulting in 15
2: STO X           // Store the result (15) into memory location X
3: SUB #15         // Subtract 15 from the accumulator, leaving it at zero
4: JMZ 6           // If the accumulator is zero, jump to line 6
5: NOP             // No operation, just a placeholder
6: HLT             // Halt the program
```

1. Copy it
2. Run it
3. Save it
4. Open a sample
5. Open from file it

Submit the code on AulaWeb

Exercise 1: Simple Addition and infinite loop

- Modify the previous program so that it **never ends** — that is, add an infinite loop to the program

Submit the code on AulaWeb

Exercise 2: Implementing an Equality Check in Assembly Language

Objective:

Your task is to write an assembly language program (Von Neumann simulator) that **checks if two numbers, X and Y, are equal**

If they are equal, the program should store the value 1 (true) in the variable Z. If they are not equal, the program should store the value 0 (false) in Z

Suggestion:

1. You will first load the value of X into the accumulator
2. Then, subtract the value of Y from the accumulator
3. If the result is zero, it means X is equal to Y, so store 1 in Z
4. If the result is not zero, it means X is not equal to Y, so store 0 in Z

Submit the code on AulaWeb

Exercise 3: Factorial Calculation

Objective: Write an assembly program (Von Neumann simulator) that calculates the factorial of a given positive integer N. The program should prompt the user for input, calculate the factorial using iterative multiplication, and store the result in a variable

Requirements:

- Input Variable: x: An integer variable that holds the input number for which the factorial will be calculated

- Output Variable: y: An integer variable that will hold the result of the factorial of N

- **Factorial Definition**:

  The factorial of N (denoted as N!) is defined as:

  $N! = N \times (N-1) \times (N-2) \times \ldots \times 1 \qquad 0! = 1$